host_agent

Initialised

User: Show me all the canals in Exeter.?

Calling tool `send_message` with args:

```
{'target': 'planning_agent', 'task_description': 'Show me all the canals in Exeter.'}
```

---

planning_agent

Initialised

The area is Exeter, so there is 1 entry for Exeter. The entity to search for is canals, and I need to find all canals in Exeter as many entries as possible.

output steps: ["Find Exeter 1 area", "Search for all canals in Exeter as many search results"]

---

host_agent

Tool `send_message` returned result: The area is Exeter, so there is 1 entry for Exeter. The entity to search for is canals, and I need to find all canals in Exeter as many entries as possible.

output steps: ["Find Exeter 1 area", "Search for all canals in Exeter as many search results"]

Calling tool `send_message` with args:

```
{'target': 'named_area', 'task_description': 'Find the area polygon for Exeter.'}
```

---

named_area_agent

Initialised

Calling tool `call_os_ngd` with args:

```
{'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_area_polygon'}
```

Tool `call_os_ngd` returned result: - Search results have been found. - Artifact generated is: exeter_area_polygon - Description: A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter. - Count of records fetched: 98

Calling tool `send_message` with args:

```
{'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'exeter_area_polygon' to find the specific area polygon for Exeter. Save the filter
```

---

data_analysis_agent

Calling tool `generate_metadata_for_artifacts` with args:

```
{'artifact_names': ['exeter_area_polygon']}
```

Tool `generate_metadata_for_artifacts` returned result: (metadata summary for 'exeter_area_polygon')

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for Exeter using multiple relevant columns for accuracy
    filtered = gdf[
        (
            (gdf['name1_text'].str.lower() == 'exeter') |
            (gdf['name1_text'].str.lower().str.contains('exeter'))
        ) & (
            (gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement'])) |
            (gdf['description'].isin(['City', 'Town', 'Suburban Area']))
        )
    ]
```

```
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter area."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the polygon(s) representing the Exeter area, filtered using name and settlement/city/town/suburban are
    return [summary, artifact_name, artifact_description, filtered]
```

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for Exeter using multiple relevant columns for accuracy
    filtered = gdf[
        (
            (gdf['name1_text'].str.lower() == 'exeter') |
            (gdf['name1_text'].str.lower().str.contains('exeter'))
        ) & (
            (gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement'])) |
            (gdf['description'].isin(['City', 'Town', 'Suburban Area']))
        )
    ]
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter area."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the polygon(s) representing the Exeter area, filtered using name and settlement/city/town/suburban are
    return [summary, artifact_name, artifact_description, filtered]

# Call the function with the provided data list
filter_exeter_area_polygon([exeter_area_polygon])
```

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for Exeter using multiple relevant columns for accuracy
    filtered = gdf[
        (
            (gdf['name1_text'].str.lower() == 'exeter') |
            (gdf['name1_text'].str.lower().str.contains('exeter'))
        ) & (
            (gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement'])) |
            (gdf['description'].isin(['City', 'Town', 'Suburban Area']))
        )
    ]
```

```
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter area."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the polygon(s) representing the Exeter area, filtered using name and settlement/city/town/suburban are
    return [summary, artifact_name, artifact_description, filtered]
```

Tool code_executor returned result: - Filtered to 1 polygon(s) for Exeter area.

Output: Filtered to 1 polygon for the Exeter area.

A new artifact named 'filtered_exeter_area_polygon' has been created, containing only the polygon(s) representing the Exeter area, filtered using the name and settlement/city/town/suburban area attributes. If you need further analysis or visualization of this area, let me know!

---

named_area_agent

The specific area polygon for Exeter has been filtered and saved as 'filtered_exeter_area_polygon'. If you need further analysis or visualization, feel free to ask!

---

host_agent

Tool `send_message` returned result: The specific area polygon for Exeter has been filtered and saved as 'filtered_exeter_area_polygon'. If you need further analysis or visualization, feel free to ask!

Calling tool `generate_metadata_for_all_artifacts` with args:

```
{}
```

Tool `generate_metadata_for_all_artifacts` returned result: - exeter_area_polygon: A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter. - filtered_exeter_area_polygon: GeoDataFrame containing only the polygon(s) representing the Exeter area, filtered using name and settlement/city/town/suburban area attributes.

Calling tool `send_message` with args:

```
{'target': 'water_network', 'task_description': "Search for all canals within the area defined by 'filtered_exeter_area_polygon'."}
```

water_network_agent

Calling tool `call_os_ngd` with args:

```
{'filters': ['canals'], 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': False, 'filename': 'exeter_canals_data'}
```

Tool `call_os_ngd` returned result: - Multiple search results have been found in multiple datasets. A summary of each is provided. - Artifacts generated are: ['waterlinkset_exeter_canals_data', 'waterlink_exeter_canals_data'] - Descriptions: ["A geopandas dataframe containing water link set data with bbox applied for the query Search for all canals within the area defined by 'filtered_exeter_area_polygon'. with no filters applied for the query (so remember no search is performed here so spurious entities present).", "A geopandas dataframe containing water link data with bbox applied for the query Search for all canals within the area defined by 'filtered_exeter_area_polygon'. with no filters applied for the query (so remember no search is performed here so spurious entities present)."] - Counts of records fetched: [11, 430]

Calling tool `send_message` with args:

```
{'target': 'data_analysis_agent', 'task_description': "Filter the artifacts 'waterlinkset_exeter_canals_data' and 'waterlink_exeter_canals_data' to identify o
```

data_analysis_agent

Calling tool `generate_metadata_for_artifacts` with args:

```
{'artifact_names': ['waterlinkset_exeter_canals_data', 'waterlink_exeter_canals_data']}
```

Tool `generate_metadata_for_artifacts` returned result: (metadata summary for 'waterlinkset_exeter_canals_data' and 'waterlink_exeter_canals_data')

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_exeter_canals_data(data: list):
    import geopandas as gpd
    # Unpack the two input GeoDataFrames
    gdf1, gdf2 = data
    # For waterlinkset_exeter_canals_data: filter where description is 'Named Canal' or name1_text is 'Exeter Canal'
```

```
    mask1 = (
        (gdf1['description'] == 'Named Canal') |
        (gdf1['name1_text'] == 'Exeter Canal')
    )
    filtered1 = gdf1[mask1]
    # For waterlink_exeter_canals_data: filter where description is 'Canal' or name1_text is 'Exeter Canal'
    mask2 = (
        (gdf2['description'] == 'Canal') |
        (gdf2['name1_text'] == 'Exeter Canal')
    )
    filtered2 = gdf2[mask2]
    # Combine the filtered results
    filtered_combined = gpd.GeoDataFrame(pd.concat([filtered1, filtered2], ignore_index=True), crs=gdf1.crs)
    summary = f"Filtered canal data from both sources. Total entries: {len(filtered_combined)}. Columns: {list(filtered_combined.columns)}."
    artifact_name = 'filtered_exeter_canals_data'
    artifact_description = 'GeoDataFrame containing only the relevant canal data from both input datasets, filtered by canal-related description and name.'
    return [summary, artifact_name, artifact_description, filtered_combined]
```

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_exeter_canals_data(data: list):
    import geopandas as gpd
    import pandas as pd
    # Unpack the two input GeoDataFrames
    gdf1, gdf2 = data
    # For waterlinkset_exeter_canals_data: filter where description is 'Named Canal' or name1_text is 'Exeter Canal'
    mask1 = (
        (gdf1['description'] == 'Named Canal') |
        (gdf1['name1_text'] == 'Exeter Canal')
    )
    filtered1 = gdf1[mask1]
    # For waterlink_exeter_canals_data: filter where description is 'Canal' or name1_text is 'Exeter Canal'
    mask2 = (
        (gdf2['description'] == 'Canal') |
        (gdf2['name1_text'] == 'Exeter Canal')
    )
    filtered2 = gdf2[mask2]
    # Combine the filtered results
    filtered_combined = gpd.GeoDataFrame(pd.concat([filtered1, filtered2], ignore_index=True), crs=gdf1.crs)
    summary = f"Filtered canal data from both sources. Total entries: {len(filtered_combined)}. Columns: {list(filtered_combined.columns)}."
    artifact_name = 'filtered_exeter_canals_data'
    artifact_description = 'GeoDataFrame containing only the relevant canal data from both input datasets, filtered by canal-related description and name.'
    return [summary, artifact_name, artifact_description, filtered_combined]
```

Tool Code Executor

```python
import matplotlib
matplotlib.use('Agg')
def filter_exeter_canals_data(data: list):
    import geopandas as gpd
    import pandas as pd
    # Unpack the two input GeoDataFrames
    gdf1, gdf2 = data
    # For waterlinkset_exeter_canals_data: filter where description is 'Named Canal' or name1_text is 'Exeter Canal'
    mask1 = (
        (gdf1['description'] == 'Named Canal') |
        (gdf1['name1_text'] == 'Exeter Canal')
    )
    filtered1 = gdf1[mask1]
    # For waterlink_exeter_canals_data: filter where description is 'Canal' or name1_text is 'Exeter Canal'
    mask2 = (
        (gdf2['description'] == 'Canal') |
        (gdf2['name1_text'] == 'Exeter Canal')
    )
    filtered2 = gdf2[mask2]
    # Reproject filtered2 to match filtered1's CRS if needed
    if filtered1.crs != filtered2.crs:
        filtered2 = filtered2.to_crs(filtered1.crs)
    # Combine the filtered results
    filtered_combined = gpd.GeoDataFrame(pd.concat([filtered1, filtered2], ignore_index=True), crs=filtered1.crs)
    summary = f"Filtered canal data from both sources. Total entries: {len(filtered_combined)}. Columns: {list(filtered_combined.columns)}."
    artifact_name = 'filtered_exeter_canals_data'
    artifact_description = 'GeoDataFrame containing only the relevant canal data from both input datasets, filtered by canal-related description and name.'
    return [summary, artifact_name, artifact_description, filtered_combined]
```

Tool code_executor returned result: - Filtered canal data from both sources. Total entries: 6. Columns: [...]

Output: Filtered canal data from both sources. Total entries: 6. Columns: [...]

An artifact has been generated with the name: filtered_exeter_canals_data and the description: GeoDataFrame containing only the relevant canal data from both input datasets, filtered by canal-related description and name.

---

water_network_agent

The search for canals within the 'filtered_exeter_area_polygon' has been completed. The relevant canal data has been filtered and saved as:

- **Artifact Name:** `filtered_exeter_canals_data`
- **Description:** GeoDataFrame containing only the relevant canal data from both input datasets, filtered by canal-related description and name.

If you need further analysis or details, feel free to ask!

---

host_agent

Calling tool `send_message` with args:

```
{'target': 'plotting_agent', 'task_description': "Plot all the canals in Exeter using the artifact 'filtered_exeter_canals_data' and overlay the Exeter area b
```

---

plotting_agent

Calling tool `generate_metadata_for_artifacts` with args:

```
{'artifact_names': ['filtered_exeter_canals_data', 'filtered_exeter_area_polygon']}
```

Tool `generate_metadata_for_artifacts` returned result: (metadata summary for 'filtered_exeter_canals_data' and 'filtered_exeter_area_polygon')

Tool Code Executor

```python
def plot_exeter_canals_with_boundary(data: list):
    import geopandas as gpd
    import folium
    from folium import GeoJson
    import os
    # Unpack data
    canals_gdf = data[0].copy()
    exeter_area_gdf = data[1].copy()

    # Ensure CRS is EPSG:4326 for folium
    if canals_gdf.crs is not None and canals_gdf.crs.to_epsg() != 4326:
        canals_gdf = canals_gdf.to_crs(epsg=4326)
    if exeter_area_gdf.crs is not None and exeter_area_gdf.crs.to_epsg() != 4326:
        exeter_area_gdf = exeter_area_gdf.to_crs(epsg=4326)

    # Filter for canals in Exeter using multiple columns for quality
    # Use 'description' for 'Canal' or 'Named Canal' and 'name1_text' for 'Exeter Canal'
    canal_mask = (
        (canals_gdf['description'].isin(['Canal', 'Named Canal'])) &
        (canals_gdf['name1_text'].str.contains('Exeter', case=False, na=False))
    )
```

```
    exeter_canals = canals_gdf[canal_mask]

    # Get centroid of Exeter area for map centering
    exeter_centroid = exeter_area_gdf.geometry.iloc[0].centroid
    m = folium.Map(location=[exeter_centroid.y, exeter_centroid.x], zoom_start=13, tiles='cartodbpositron')

    # Plot Exeter area boundary
    folium.GeoJson(
        exeter_area_gdf.geometry.iloc[0],
        name='Exeter Area',
        style_function=lambda x: {'color': 'blue', 'weight': 2, 'fillOpacity': 0.05}
    ).add_to(m)

    # Plot canals
    for _, row in exeter_canals.iterrows():
        folium.GeoJson(
            row['geometry'],
            name=row.get('name1_text', 'Canal'),
            style_function=lambda x: {'color': 'green', 'weight': 4}
        ).add_to(m)

    folium.LayerControl().add_to(m)

    # Save map
    map_filename = 'exeter_canals_with_boundary.html'
    m.save(map_filename)

    # Prepare summary
    summary_rows = exeter_canals[['name1_text', 'description', 'geometry_length', 'geometry_length_m']].head(5)
    summary = f"Plotted {len(exeter_canals)} Exeter canal features over the Exeter area boundary. First 5: {summary_rows.to_dict(orient='records')}"
    artifact_name = 'exeter_canals_with_boundary_map'
    artifact_description = 'Folium map showing all Exeter canals (filtered by description and name1_text) overlaid with the Exeter area boundary.'
    return [summary, artifact_name, artifact_description, map_filename]
```

Tool code_executor returned result: - Plotted 3 Exeter canal features over the Exeter area boundary. First 5: [{'name1_text': 'Exeter Canal', 'description': 'Named Canal', 'geometry_length': 8280.967199, 'geometry_length_m': nan}, {'name1_text': 'Exeter Canal', 'description': 'Canal', 'geometry_length': nan, 'geometry_length_m': 1846.551}, {'name1_text': 'Exeter Canal', 'description': 'Canal', 'geometry_length': nan, 'geometry_length_m': 18.632}]

Artifact name: exeter_canals_with_boundary_map

Artifact description: Folium map showing all Exeter canals (filtered by description and name1_text) overlaid with the Exeter area boundary.

Artifact data: exeter_canals_with_boundary.html

---

host_agent

All the canals in Exeter have been found and plotted. The map includes:

- All canal features in Exeter (from the water network database)
- The Exeter area boundary for context

Summary of results: - 3 canal features were found, all named "Exeter Canal" with varying lengths.

If you would like to view the map or need further details about the canals (such as their attributes or a list), please let me know!